



Introduction to AgentOps on Google Cloud

Google Cloud

Proprietary & Confidential

1

Welcome to the first module: Introduction to AgentOps on Google Cloud.

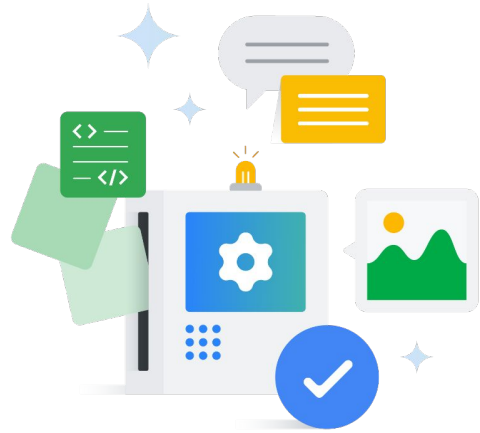
In this module, we'll introduce the operational discipline required to run AI agents reliably in production environments.

Many organizations are quickly building agents, but the real challenge is operating them at scale. Agents introduce new operational challenges compared to traditional applications or even standard machine learning systems.

AgentOps is the emerging practice that applies DevOps and MLOps principles specifically to AI agents.

In this module, you learn to ...

- 01 Navigate the challenges of managing production agents
- 02 Define the core principles of AgentOps
- 03 Architect agent operations on Google Cloud



In this module, we focus on three main learning objectives.

First, we'll explore the **challenges involved in managing production AI agents**. Agents introduce new complexities such as non-deterministic outputs, tool orchestration, and unpredictable usage patterns.

Second, we'll define the **core principles of AgentOps**. These principles guide how we build reliable, observable, and secure agent systems.

Finally, we'll examine how to **architect an AgentOps platform using Google Cloud services**.

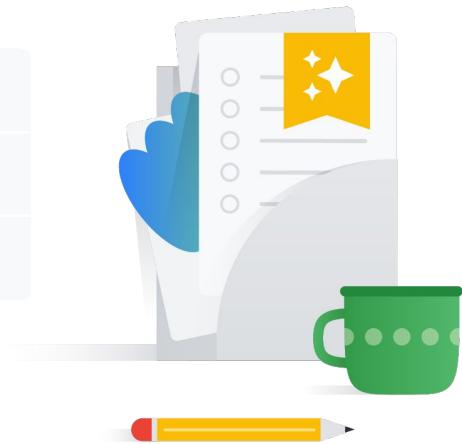
By the end of this module, you should understand the operational foundations needed to run agents in production.

Topics

01 Challenges of managing production agents

02 Core principles of AgentOps

03 AgentOps on Google Cloud



Here's our agenda. We'll begin with **operational challenges of production agents**.

Next, we'll define the **core principles of AgentOps**, which provide the operational framework for managing agents.

Finally, we'll map those principles to **Google Cloud tools and services**, showing how they can be implemented in practice.

Challenges of managing production agents

- Efficiency and consistency
 - Manual deployments are slow and error-prone
- Scale
 - Inconsistent environments lead to unpredictable behavior
- Quality
 - Reproducibility is key for debugging and rollbacks
- Security
 - Automation reduces toil and improves release velocity
- Observability
- Cost management

Let's begin with the first challenge: scaling agent systems.

Deploying agents manually quickly becomes impractical. Manual deployments are slow and error-prone, especially when multiple environments are involved.

Consistency between environments is critical. If development, staging, and production environments differ, agents may behave unpredictably.

Automation becomes essential to maintain velocity while minimizing operational risk.

As organizations deploy more agents, they must manage them the same way they manage distributed applications—with automated pipelines, infrastructure management, and reproducibility.

Challenges of managing production agents

- Efficiency and consistency
 - Scale
 - Quality
 - Security
 - Observability
 - Cost management
- Agent usage can be unpredictable and bursty
 - Infrastructure must scale to meet demand without over-provisioning
 - Managing multiple agent versions and environments becomes complex
 - Global deployments require regional considerations

Another key challenge is handling unpredictable demand.

Agent workloads can be highly variable. Some agents experience bursty traffic, especially when tied to user-facing applications.

Infrastructure must scale dynamically to handle these workloads without over-provisioning resources.

Additionally, organizations often run multiple agent versions simultaneously across development, staging, and production environments.

Global deployments also introduce regional considerations such as latency, compliance requirements, and data locality.

Challenges of managing production agents

- Efficiency and consistency
 - Scale
 - Quality
 - Security
 - Observability
 - Cost management
- Agent responses can be non-deterministic
 - Defining 'good' is subjective and use-case specific
 - Need for automated evaluation to prevent regressions
 - Hallucinations, grounding, and safety are critical concerns

Quality is particularly difficult to manage in agent systems.

Unlike traditional software, agent outputs are non-deterministic. The same prompt may produce slightly different responses each time.

Defining what “good” looks like can be subjective and depends heavily on the use case.

To maintain quality, organizations need automated evaluation systems that test agents against benchmark datasets and real-world scenarios.

Additional risks such as hallucinations, incorrect grounding, and unsafe outputs must also be managed carefully.

Challenges of managing production agents

- Efficiency and consistency
- Scale
- Quality
- Security
 - Prompt injection and jailbreaking risks
 - Data exfiltration through agent tools or responses
 - Overly permissive agent permissions (IAM)
 - Securing the entire agentic system (UI, agent, tools, data)
- Observability
- Cost management

Security is a critical concern when deploying agents in production.

Agentic systems introduce new attack surfaces.

For example, prompt injection attacks attempt to manipulate the agent's instructions. Jailbreaking attempts try to bypass safety restrictions.

Agents may also have access to external tools and databases, which introduces the risk of data exfiltration if permissions are not properly controlled.

To mitigate these risks, organizations must implement strong IAM policies, guardrails, and security monitoring across the entire agent ecosystem.

Challenges of managing production agents

- Efficiency and consistency
 - Scale
 - Quality
 - Security
 - Observability
 - Cost management
- Understanding the agent's 'thought process' (trajectory)
 - Debugging complex chains of tool calls and model interactions
 - Monitoring for performance, cost, and quality degradation
 - Correlating user feedback with specific agent traces

Observability becomes much more complex with agent systems.

Agents often interact with multiple components:

- models
- external tools
- databases
- APIs

Understanding an agent's decision-making path—sometimes called the trajectory—is essential for debugging.

Teams need the ability to trace:

- prompts
- tool calls
- intermediate responses
- final outputs

Without proper observability, diagnosing production issues becomes extremely difficult.

Challenges of managing production agents

- Efficiency and consistency
 - LLM token costs can be significant and hard to predict
- Scale
 - Inefficient prompts lead to wasted spend
- Quality
 - Balancing performance vs cost for model selection
- Security
 - Attributing costs to specific users, teams, or features
- Observability
- Cost management

Cost management is another important challenge.

Large language models are typically billed based on token usage, which means costs can grow quickly and unpredictably.

Poorly designed prompts or inefficient workflows can lead to unnecessary token consumption.

Organizations must balance performance with cost by selecting appropriate models, optimizing prompts, and monitoring usage in real time.

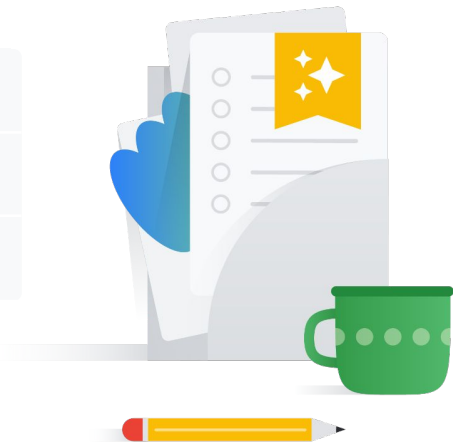
Cost attribution is also important so that teams understand which features or services are driving expenses.

Topics

01 Challenges of managing production agents

02 Core principles of AgentOps

03 AgentOps on Google Cloud



Now that we've discussed the operational challenges of agent systems, we can introduce the framework designed to address them: AgentOps.

AgentOps provides a structured set of practices that help organizations build, deploy, and operate agents reliably.

Next, we'll explore the core principles that define AgentOps.

Introducing Agent Operations (AgentOps)

Applying DevOps/MLOps principles to agents

- ✓ Goal: Build, test, and operate agents reliably and at scale
- ✓ Focuses on the entire agent lifecycle
- ✓ Emphasizes automation, evaluation, and observability

AgentOps = DevOps + MLOps tailored for AI Agents

AgentOps extends established operational practices from DevOps and MLOps into the world of AI agents.

The goal is to manage the entire lifecycle of an agent system, from development to deployment and ongoing monitoring.

AgentOps emphasizes three key capabilities:

- Automation, so agents can be deployed reliably.
- Evaluation, to ensure that quality remains high over time.
- And observability, so teams can understand agent behavior and quickly diagnose issues.

What is AgentOps?

Agent Operations (AgentOps) is a comprehensive operational framework for the **management** and **maintenance** of agent systems. It defines the necessary **lifecycle management** for **production-grade** agentic systems, extending **MLOps** concepts to agent-specific needs.

AgentOps can be defined as a comprehensive operational framework for agent systems.

It builds on MLOps concepts but addresses additional complexities unique to agentic architectures.

These include multi-step reasoning workflows, tool orchestration, and dynamic model interactions.

AgentOps provides the lifecycle management practices required to operate production-grade agents reliably.

Core principles of AgentOps

- ✓ Automation and repeatability
 - Automate everything: build, test, evaluation, and deployment
- ✓ Comprehensive observability
 - Use Infrastructure as Code (IaC) for environments
- ✓ Continuous evaluation and feedback
 - Create reproducible processes to reduce manual errors
- ✓ Security and guardrails by design
 - Goal: Fast, reliable, and consistent delivery of agent updates
- ✓ Cost visibility and management (FinOps)

The first core principle of AgentOps is automation and repeatability.

Agent deployments should be fully automated—from building containers to running tests and deploying infrastructure.

Infrastructure as Code tools allow teams to define environments programmatically, ensuring consistency across environments.

Automation reduces manual errors and enables faster and more reliable delivery of agent updates.

Core principles of AgentOps

- ✓ Automation and repeatability
- ✓ Comprehensive observability
 - If you can't see it, you can't fix it
 - Implement structured logging, distributed tracing, and monitoring
 - Track agent performance, cost, and behavior
 - Use observability data to debug issues and identify improvements
- ✓ Continuous evaluation and feedback
- ✓ Security and guardrails by design
- ✓ Cost visibility and management (FinOps)

The second principle is comprehensive observability.

If you can't observe your system, you can't effectively operate it.

Agent systems require logging, tracing, and monitoring capabilities to capture:

- prompts
- responses
- tool usage
- performance metrics
- and costs.

This observability data is essential for debugging issues and identifying opportunities for improvement.

Core principles of AgentOps

- ✓ Automation and repeatability
 - ✓ Comprehensive observability
 - ✓ Continuous evaluation and feedback
 - ✓ Security and guardrails by design
 - ✓ Cost visibility and management (FinOps)
- Evaluation is not a one-time task; it's continuous
 - Combine automated evaluation with human-in-the-loop feedback
 - Use evaluation metrics to gate promotions to production
 - Analyze production data to create a feedback loop for improvement

Evaluation must be continuous rather than a one-time testing activity.

Agents should be evaluated regularly using automated evaluation frameworks and benchmark datasets.

Human feedback can also play an important role in assessing agent performance.

Evaluation metrics should be used to determine whether new versions of an agent are ready to be promoted to production.

This creates a continuous improvement loop for the system.

Core principles of AgentOps

- ✓ Automation and repeatability
 - ✓ Comprehensive observability
 - ✓ Continuous evaluation and feedback
 - ✓ Security and guardrails by design
 - ✓ Cost visibility and management (FinOps)
- Security is not an afterthought
 - Implement guardrails for inputs (sanitization) and outputs (filtering)
 - Apply the principle of least privilege to agent permissions
 - Continuously scan for vulnerabilities and audit agent activity

Security must be designed into the system from the beginning.

Guardrails should be applied both to agent inputs and outputs.

Input validation can help detect malicious prompts, while output filtering can prevent unsafe or sensitive responses.

Additionally, the principle of least privilege should be applied when assigning permissions to agents.

Regular audits and vulnerability scans help maintain system security over time.

Core principles of AgentOps

- ✓ Automation and repeatability
 - ✓ Comprehensive observability
 - ✓ Continuous evaluation and feedback
 - ✓ Security and guardrails by design
 - ✓ Cost visibility and management (FinOps)
- Understand the cost components of your agent
 - Monitor costs in real-time and set budget alerts
 - Attribute costs to specific features or teams
 - Actively pursue cost optimization strategies

The final principle is cost visibility and management, often referred to as FinOps.

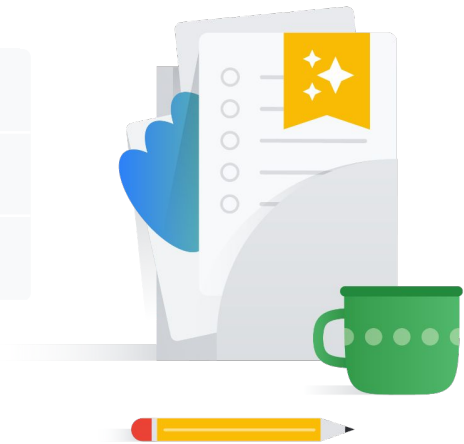
Organizations must understand the cost drivers of their agent systems.

Monitoring tools should provide real-time visibility into model usage and token consumption.

Teams can then attribute costs to specific features or teams and optimize workloads to reduce unnecessary spending.

Topics

- 01 Challenges of managing production agents
- 02 Core principles of AgentOps
- 03 AgentOps on Google Cloud



Now that we understand the principles of AgentOps, the next step is to see how these principles can be implemented using Google Cloud services.

Google Cloud provides a comprehensive set of tools that support automation, observability, security, evaluation, and cost management.

Let's look at how these components fit together.

Mapping AgentOps principles to Google Cloud services or tools

AgentOps Principle	Google Cloud Service or Tool
Automation	Cloud Build, Cloud Deploy, Terraform
Observability	Cloud Monitoring, Cloud Logging, Cloud Trace, Data Studio
Evaluation	Agent Platform Evaluation, ADK Eval, Cloud Logging
Security	IAM, Model Armor, Sensitive Data Protection, Secret Manager
FinOps	Cloud Billing, Cloud Monitoring

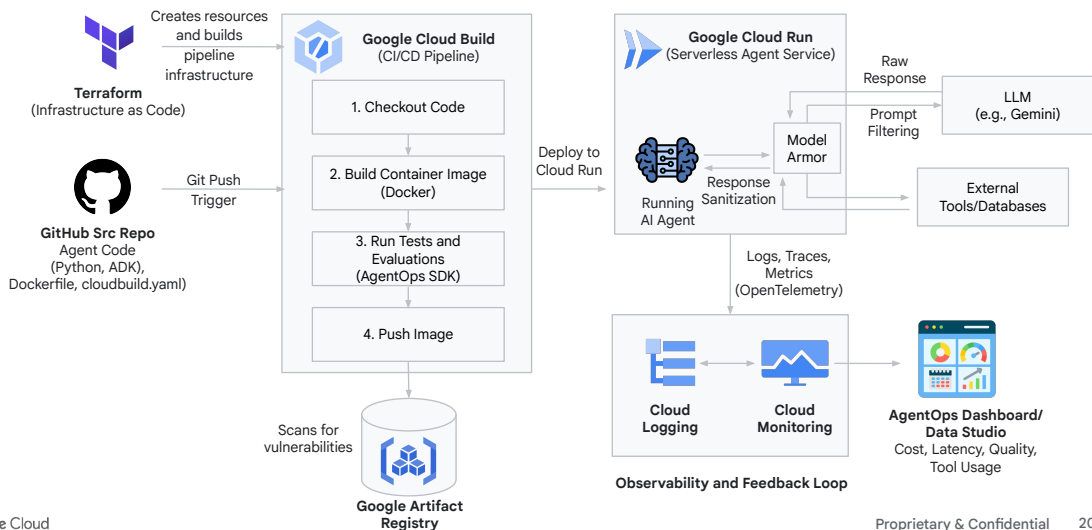
This slide maps the AgentOps principles we discussed to specific Google Cloud services.

For example:

- Automation can be implemented using tools like Cloud Build, Cloud Deploy, and Terraform.
- Observability can be handled through Cloud Logging, Cloud Monitoring, and Cloud Trace.
- Evaluation can be supported by Agent Platform evaluation service and the ADK Eval tool
- Security capabilities include IAM, Secret Manager, and Model Armor.
- Finally, cost visibility is provided through Cloud Billing and monitoring tools.

Together, these services form the foundation of an AgentOps platform.

AgentOps on Google Cloud



This diagram shows a reference architecture for running agents on Google Cloud.

The workflow typically begins with code stored in a source repository like GitHub. When code changes are pushed, a CI/CD pipeline such as Cloud Build is triggered. The pipeline builds the container image, runs tests and evaluations, and pushes the image to Artifact Registry. The agent service itself runs on

Cloud Run, providing a scalable serverless execution environment. Security guardrails like Model Armor help filter prompts and sanitize responses.

Meanwhile, observability tools capture logs, metrics, and traces. These signals feed dashboards where teams can monitor cost, latency, quality, and system health.

Questions and answers



This concludes the main portion of the module.

Before we move on, let's pause for questions.



Thank you.
Module 1 is complete.